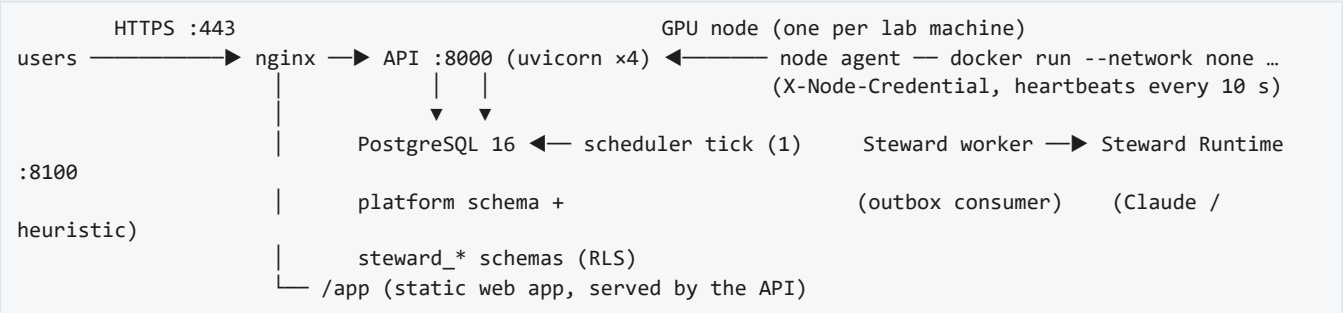
Prayog — Deployment Documentation

CS455 Deliverable 3 · Group 3 · 6 November 2026

1. What gets deployed

Prayog is five kinds of process around one PostgreSQL 16 database. Nothing else holds state: artifacts live on a filesystem path (or S3-compatible store), everything else is in the database.

Process	Command (from `backend/`)	Instances	State it holds	If it stops
API (FastAPI + web app)	`uvicorn main:app --host 127.0.0.1 --port 8000 --workers 4`	1 VM, *N* workers	none (stateless)	users and node agents get connection errors; nothing is lost (failure drill F2)
Scheduler tick (reaper + safety-net pass)	`python -m logic.scheduler.run`	**exactly 1**	none	jobs are still placed by event-driven passes, but **dead nodes are not detected** until it restarts (drill F4)
Steward worker	`python -m integration.steward_worker`	1	none	diagnoses wait in the outbox and are processed when it returns
Steward Runtime (model adapter)	`uvicorn steward_runtime.app:app --port 8100`	1	none; no DB credentials (ADR-S01)	the worker degrades to "no diagnosis available" (PRAYOG-102)
Node agent	`python -m node_agent.agent --server https://… --units 8 --state-dir /var/lib/prayog-agent --runner docker --image prayog-ml-runtime:python3.12`	1 per GPU node	its state file (running allocations, credential)	the reaper marks the node DOWN after `heartbeat_timeout` (30 s) and requeues its jobs (drill F1)
PostgreSQL 16	managed service or `docker compose up -d postgres`	1 primary (+ standby)	everything	API answers **503 + Retry-After** and recovers by itself when it returns (drill F3, after fix D3-06)



2. Environments

Environment	Where	Database	Purpose	Status
Development	each member's laptop	local PostgreSQL (`docker compose up -d postgres` or a portable cluster on 55432)	coding, unit tests	in use
CI	GitHub Actions, `ubuntu-24.04` / `ubuntu-latest`	`postgres:16` service container	migrations from empty, `alembic check`, full test suite, Docker worker image, node-agent container tests, API-to-worker e2e	green on PR #26 (commit `cc209d9`)

Environment	Where	Database	Purpose	Status
Staging / demo	one Linux VM (2 vCPU, 4 GB) + one GPU node	PostgreSQL 16 on the VM	demonstration 8–13 Nov	runbook below; see section 9 for what is verified
Production (IITK)	institute VM behind the IITK reverse proxy; lab-owned GPU nodes	managed PostgreSQL or a VM with a streaming standby	real users	not deployed (course project)

3. Two ways to deploy

Route	What it is	Status
A. Docker Compose (section 3A)	One host: PostgreSQL, a one-shot migration, the API with four workers, the tick	**Run and verified locally on 6 Nov**: this stack is what the failure, load and concurrency experiments ran against
B. systemd and nginx (section 4)	Separate processes on a VM behind nginx	Written, **not exercised**

3A. Docker Compose (verified)

`backend/Dockerfile` builds one image for the API and the tick. It runs as an unprivileged user (uid 10001) and has a health check. `docker-compose.deploy.yml` does not publish PostgreSQL to the host, refuses to start without its secrets, applies migrations before the API starts, and binds the API to `127.0.0.1:8000` unless told otherwise.

```
git clone https://github.com/Dishaswamy/Prayog.git && cd Prayog
cat > .env <<EOF
POSTGRES_USER=prayog
POSTGRES_PASSWORD=$(python3 -c "import secrets; print(secrets.token_urlsafe(24))")
POSTGRES_DB=prayog
JWT_SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_urlsafe(48))")
STEWARD_ENABLED=false
EOF
chmod 600 .env
docker compose -f docker-compose.deploy.yml up -d --build
curl -s http://127.0.0.1:8000/health            # {"status":"ok","database":"ok"}
docker compose -f docker-compose.deploy.yml exec api python -m scripts.create_admin admin@iitk.ac.in
'<strong password>'
```

Sizing. Each API worker has its own connection pool, so the compose file sets `PRAYOG_DB_POOL_SIZE=15` and `PRAYOG_DB_MAX_OVERFLOW=5`: 4 × 20 stays under PostgreSQL's 100 connections, with room for the tick and an admin session. The code's defaults (20 + 20, sized for one process) would allow 160 and hit that limit under load.

Steward. `STEWARD_ENABLED=false` runs the platform without it. To enable it, run `python -m scripts.setup_steward` once and add the worker and Runtime services (section 1).

3B. On a cloud VM (AWS EC2; not yet executed)

The course requires a cloud deployment, and **this step needs a team member's cloud account**. It should take about 20 minutes:

1. Launch an Ubuntu 24.04 `t3.medium` in `ap-south-1` with a 30 GB disk (Costing, option C). Set an AWS billing alarm at $10 first.
2. Security group: allow SSH from your own IP only, and 443 from the demo audience. Do not open 5432 or 8000.
3. Install Docker: `curl -fsSL https://get.docker.com | sudo sh && sudo usermod -aG docker ubuntu`, then log in again.
4. Run the commands in section 3A on the VM.
5. Put TLS in front. Caddy is the shortest route, and `<public-ip>.sslip.io` gives a usable name without buying a domain: `caddy reverse-proxy --from <public-ip>.sslip.io --to 127.0.0.1:8000`.

6. Run the smoke checklist (section 5), and save its output and the `/health` response as evidence. Record the actual cost in the Costing document.
7. Stop the VM between sessions. Terminate it after the demo.

3C. Prerequisites for route B

- Linux host (Ubuntu 24.04 LTS tested in CI) with Python 3.12, Git, and outbound HTTPS for `pip`.
- PostgreSQL 16 reachable from the API host; a database `prayog` and an owner role `prayog`.
- On every GPU node: Docker Engine (or rootless Docker), NVIDIA Container Toolkit, Python 3.12 and the `prayog-ml-runtime` image built from `backend/worker/Dockerfile`. The agent starts containers with `--pull=never`, so the image must already be on the node.
- A TLS certificate for the public host name (the IITK proxy or Let's Encrypt).
- Secrets generated per environment — never reused between environments, never committed: `JWT_SECRET_KEY` (`python -c "import secrets; print(secrets.token_urlsafe(48))"`), `POSTGRES_PASSWORD`, `STEWARD_DB_PASSWORD` (must differ), optionally `ANTHROPIC_API_KEY` and `SMTP_*`.

4. First deployment (staging VM)

```
# 4.1 code and environment
sudo useradd --system --create-home --home-dir /opt/prayog prayog
sudo -u prayog git clone https://github.com/Dishaswamy/Prayog.git /opt/prayog/app
cd /opt/prayog/app && git checkout <release tag or main commit>
sudo -u prayog python3.12 -m venv /opt/prayog/venv
sudo -u prayog /opt/prayog/venv/bin/pip install -r backend/requirements.txt -e ./steward
sudo -u prayog cp .env.example .env && sudo chmod 600 .env      # then edit: secrets, POSTGRES_HOST,
STEWARD_PUBLIC_BASE_URL=https://<host>

# 4.2 database
cd backend
/opt/prayog/venv/bin/python -m alembic upgrade head          # platform tables
/opt/prayog/venv/bin/python -m alembic check                 # must print "No new upgrade operations
detected"
/opt/prayog/venv/bin/python -m scripts.setup_steward         # Steward role: NOSUPERUSER NOBYPASSRLS,
schemas, grants
/opt/prayog/venv/bin/python -m scripts.create_admin admin@iitk.ac.in '<strong password>'
# Labs have no API yet: insert them once with SQL (see README "Status"), then register nodes as admin.
```

Never run `scripts.seed_dev` or `scripts.seed_gpu_demo` on a shared deployment: their passwords are public by design.

4.3 Services (systemd)

One unit per process, each restarting itself. The tick **must** be supervised: drill F4 showed that while it is down no node is ever declared DOWN.

```
# /etc/systemd/system/prayog-api.service
[Unit]
Description=Prayog API
After=network-online.target
[Service]
User=prayog
WorkingDirectory=/opt/prayog/app/backend
EnvironmentFile=/opt/prayog/app/.env
ExecStart=/opt/prayog/venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000 --workers 4 --proxy-headers
Restart=always
RestartSec=2
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/lib/prayog/artifacts
[Install]
WantedBy=multi-user.target
```

`prayog-tick.service`, `prayog-steward-worker.service` and `prayog-steward-runtime.service` are the same with `ExecStart` replaced by the commands in section 1 (the Runtime gets no `EnvironmentFile` entries for database credentials). Enable all four: `sudo systemctl enable --now prayog-api prayog-tick prayog-steward-worker prayog-steward-runtime`.

Sizing the API. The load test (Performance report section 4) found that one uvicorn process with SQLAlchemy's default pool (5 + 10 overflow) times out after 30 s once more than ~15 requests hold a connection, and that one account's logins are serialised by its row lock. With `--workers 4` each worker has its own pool: set `PRAYOG_DB_POOL_SIZE=15` and `PRAYOG_DB_MAX_OVERFLOW=5`, so that `4 × 20 = 80` stays below PostgreSQL's `max_connections` (default 100) minus the tick, worker and admin sessions.

4.4 Reverse proxy (nginx)

```
server {
    listen 443 ssl http2;
    server_name prayog.example.iitk.ac.in;
    ssl_certificate     /etc/ssl/prayog/fullchain.pem;
    ssl_certificate_key /etc/ssl/prayog/privkey.pem;
    client_max_body_size 300m;                      # artifact uploads; the API enforces its own limits
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;
    add_header Referrer-Policy same-origin always;
    add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'none'" always;
    add_header Strict-Transport-Security "max-age=31536000" always;
    location /metrics { allow 10.0.0.0/8; deny all; proxy_pass http://127.0.0.1:8000; }   # Prometheus only
    location / { proxy_pass http://127.0.0.1:8000; proxy_set_header X-Forwarded-For $remote_addr;
                 proxy_set_header X-Forwarded-Proto https; proxy_read_timeout 120s; }
}
```

The API already sends CSP, `nosniff`, `X-Frame-Options` and `Referrer-Policy` on `/app` (adversarial probe A21); the proxy adds them to every other path and adds HSTS, which only makes sense behind TLS. Set `STEWARD_COOKIE_SECURE=true` once TLS terminates at the proxy.

4.5 GPU nodes

1. Admin registers the node: `POST /admin/nodes` (or the admin web screen) → a one-time enrolment token.
2. On the node, as a dedicated `prayog-agent` user in the `docker` group: `python -m node_agent.agent --server https://<host> --units <n> --state-dir /var/lib/prayog-agent --runner docker --image prayog-ml-runtime:python3.12 --enrolment-token <token>`
3. The first heartbeat moves the node `PENDING_AGENT → AVAILABLE`. Later starts reuse the stored credential; run the agent under systemd with `Restart=always` as for the API.
4. Keep `/var/lib/prayog-agent` mode `0700` (after fix D3-05 each run workspace is `0700` as well).

5. Verification after deployment (smoke checklist)

#	Check	Expected
1	`curl https://<host>/health`	`{"status":"ok","database":"ok"}`
2	`alembic current`	the newest revision of `backend/alembic/versions`
3	Sign in as admin in the web app	admin workspace with the pool picture
4	A node agent started	node `AVAILABLE` within one heartbeat
5	Submit `{"gpus":1,"walltime":"00:05"}` as an approved test user	`IN_PROCESS → QUEUED → SCHEDULED → RUNNING → COMPLETED`
6	Stop the agent for 60 s	node `DOWN`, a `NODE_DOWN` alert, the job back in `QUEUED` with its original submit time

#	Check	Expected
7	`curl -H 'Origin: https://evil.example' -X OPTIONS https://<host>/jobs`	no `Access-Control-Allow-Origin`
8	`curl -I https://<host>/app/`	CSP, `nosniff`, `X-Frame-Options: DENY`, HSTS

6. Upgrades and rollback

1. Announce a window. A restart of the API takes seconds, and a longer outage no longer evicts running jobs: the reaper judges heartbeats only once the API has been back for a full `heartbeat_timeout` (fix D3-08, Failure report section 5). Node agents keep their runs going and retry.
2. `pg_dump -Fc prayog > prayog-$(date +%F).dump` (see section 7).
3. `git fetch && git checkout <new commit>`; `pip install -r backend/requirements.txt -e ./steward`.
4. `alembic upgrade head`, then `alembic check`.
5. `systemctl restart prayog-api prayog-steward-worker prayog-steward-runtime`, then `systemctl start prayog-tick`.
6. Run the smoke checklist (section 5).

Rollback: `git checkout <previous commit>`, `alembic downgrade <previous revision>` if the release added a migration (every migration in `backend/alembic/versions` defines `downgrade`; the two merge revisions are no-ops; data in a dropped column is lost), restart the services. If a migration cannot be downgraded safely, restore the dump from step 2.

7. Backups and recovery objectives

Item	Method	Frequency	Retention
Database	`pg_dump -Fc` to a separate disk, plus WAL archiving (or the managed service's point-in-time recovery)	nightly + continuous WAL	14 days
Artifact store (`PRAYOG_ARTIFACT_ROOT`)	`rsync` / object-store versioning; artifacts are content-addressed (BLAKE3/SHA-256), so an incremental copy is safe	nightly	30 days
`.env` and node credentials	offline, in the team's password manager	on change	—

Targets for the institute deployment: RPO ≤ 24 h for artifacts and ≤ 5 min for the database (WAL), RTO ≤ 1 h. A restore drill (restore the dump into a scratch database, run `alembic check` and the smoke checklist) belongs on the demo-week checklist; it has not been run yet.

8. Monitoring and operations

- **Liveness:** `GET /health` every 30 s from the proxy or Uptime Kuma; it returns 503 when the database is unreachable.
- **Metrics:** `GET /metrics` (Prometheus text: `prayog_queue_depth`, `prayog_free_units`, `prayog_pass_duration_seconds`, `prayog_claim_failures_total`, `prayog_dispatch_timeouts_total`, …). Restrict it to the monitoring network (section 4.4): it is unauthenticated, although it carries counts only.
- **Logs:** journald per unit. Alert on `prayog.scheduler` ERROR lines (after fix D3-03 a refused claim logs "refused by the allocation indexes"), on `database unavailable` (fix D3-06), and on `NODE_DOWN` alerts in the admin screen.
- **Audit:** `audit_events` is append-only (a trigger refuses UPDATE/DELETE); sign-ins, lockouts, approvals and admin actions are there.

9. What has and has not been verified

Statement	Evidence
Migrations apply to an empty PostgreSQL 16 and leave no pending changes	CI step "Apply migrations to clean PostgreSQL" + "Verify schema has no pending migration"; repeated locally on 6 Nov for three fresh databases
The worker image builds and runs real hardened containers (`--network none`, read-only, `--cap-drop ALL`, uid 65534)	CI steps "Build isolated worker runtime image", "Run real Docker worker smoke tests", "Run GPU node agent tests in the real hardened container"
API, tick, simulated node agent and PostgreSQL run together and recover from crashes	live failure drills F1–F4 on 6 Nov, and `backend/experiments/failure_experiment.py` at production timeouts (Failure and Recovery report)
The Docker image builds; the compose stack starts from empty, migrates, serves `/health`, runs as a non-root user	run repeatedly on 6 Nov: every experiment started from `docker compose down -v` and `up`
Deployment on a cloud service (course section 16)	**not done**: needs a team member's cloud account (section 3B). Jira item PRAYOG-24 is marked Done but this is not evidence for it
systemd units, nginx configuration, a real GPU, TLS, backups and restore	**not verified** — written for the demo deployment, to be exercised in Sprint 3